

1. Advance Python Programming

a. Introduction to the print() function in Python.

- A. The print() function in Python is one of the most basic and commonly used functions. It is used to display output on the screen (console).

Ex : `print("Hello, World!")`

b. Formatting outputs using f-strings and format().

- A. Formatting output in Python helps you present data in a clean, readable, and professional way. Two commonly used methods are f-strings and the format() method

Ex : `a = 5`

`b = 3`

`print(f"Sum is {a + b}")`

2. Reading Data from Keyboard

a. Formatting outputs using f-strings and format().

- A. Reading data from the keyboard in Python is done using the built-in input() function. It allows your program to accept values entered by the user at runtime.

Ex : `name = input("Enter your name: ")`

`print(f"Hello, {name}!")`

b. Converting user input into different data types (e.g., int, float, etc.).

- A. When you take input using Python's input() function, the value is always returned as a string (str). To perform calculations or logical operations, you often need to convert it into other data types like int, float, etc.

Ex : `a = int(input("Enter a number: "))`

`b = int(input("Enter another number: "))`

`print(a + b)`

3. Opening and Closing Files

a. Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

- A. In Python, when you open a file using open(), you must specify a mode that tells Python what kind of operation you want to perform—reading, writing, or both.

1. 'r' is for Read Mode

- a. Opens a file for reading only
- b. File must exist, otherwise error occurs

Ex : `file = open("data.txt", "r")`

`print(file.read())`

`file.close()`

2. 'w' is for Write Mode

- a. Opens a file for **writing**
- b. Creates a new file if it doesn't exist
- c. **Overwrites** existing content

```
Ex :   file = open("data.txt", "w")
        file.write("Hello World")
        file.close()
```

3. 'a' is for Append Mode

- a. Adds new content at the **end of the file**
- b. Does not delete existing data

```
Ex : file = open("data.txt", "a")
        file.write("\nNew line added")
        file.close()
```

4. 'r+' is for Read & Write Mode

- a. Opens file for **both reading and writing**
- b. File must exist
- c. Writing starts from the beginning (may overwrite content)

```
Ex : file = open("data.txt", "r+")
        print(file.read())
        file.write("Updated text")
        file.close()
```

5. 'w+' is for Write & Read Mode

- a. Opens file for **writing and reading**
- b. Creates file if not exists
- c. **Erases existing content first**

```
Ex :   file = open("data.txt", "w+")
        file.write("New content")
        file.seek(0) # Move cursor to beginning
        print(file.read())
        file.close()
```

b. Using the open() function to create and access files.

- A. In Python, the open() function is used to create, open, and access files for reading or writing

Opening a File

```
file_object = open("filename", "mode")
filename → Name (or path) of the file
mode → Type of operation ('r', 'w', 'a', etc.)
```

Accessing (Reading) a File

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

c. Closing files using close().

A. close() is used to terminate the connection between your program and the file.

Ex : file.close()

4. Reading and Writing Files

a. Reading from a file using read(), readline(), readlines().

A. Reading data from a file in Python is done after opening it in read mode ('r').

Python provides three main methods: read(), readline(), and readlines().

1. Using read()

- Reads the entire file content as a single string.
with open("data.txt", "r") as file:
 content = file.read()
 print(content)

2. Using readline()

- Reads one line at a time
with open("data.txt", "r") as file:
 line1 = file.readline()
 line2 = file.readline()
 print (line1)
 print(line2)

3. Using readlines()

- Reads all lines into a list
with open("data.txt", "r") as file:
 lines = file.readlines()
 print(lines)

5. Exception Handling

a. Introduction to exceptions and how to handle them using try, except, and finally.

A. In Python, an exception is an error that occurs during program execution and disrupts the normal flow of the program. Instead of crashing, we can handle these errors using exception handling.

- We use try to test code and except to handle errors.
- finally block always runs whether an error occurs or not

Ex : try:

```
    a = int(input("Enter number: "))  
    b = int(input("Enter another number: "))
```

```

        result = a / b
    except ZeroDivisionError:
        print("Cannot divide by zero!")
    except ValueError:
        print("Invalid input!")
    Else:
        print(f"Result is {result}")
    Finally:
        print("Program ended.")

```

b. Understanding multiple exceptions and custom exceptions.

A. Handling errors well is what makes your Python programs robust and professional. Let's break this into two parts: multiple exceptions and custom exceptions.

1. Multiple except Blocks

Ex : try:

```

        a = int(input("Enter a number: "))
        b = int(input("Enter another number: "))
        print(a / b)

```

```

    except ValueError:
        print("Invalid input! Please enter numbers.")

```

```

    except ZeroDivisionError:
        print("Cannot divide by zero.")

```

2. Single except with Multiple Exceptions

Ex : try:

```

        a = int(input("Enter a number: "))
        b = int(input("Enter another number: "))
        print(a / b)

```

```

    except (ValueError, ZeroDivisionError):
        print("An error occurred!")

```

3. Catch All Exceptions (Not Recommended)

Ex : try:

```

        x = int("abc")
    except Exception as e:
        print("Error:", e)

```

4. Custom Exceptions

```
Ex :   class InvalidAgeError(Exception):
        pass

    try:
        age = int(input("Enter age: "))

        if age < 18:
            raise InvalidAgeError("Age must be 18+")

        print("Access granted")

    except InvalidAgeError as e:
        print("Error:", e)
```

6. Class and Object (OOP Concepts)

a. Understanding the concepts of classes, objects, attributes, and methods in Python.

A. Object-Oriented Programming (OOP) in Python is built around four core ideas: classes, objects, attributes, and methods. Understanding these will help you design clean, reusable code.

1. Class (Blueprint)

A class is like a blueprint or template used to create objects.

```
Ex :   class Student:
        Pass
```

This defines a class named Student, but it doesn't do anything yet.

2. Object (Instance)

An object is an instance of a class.

```
Ex :

s1 = Student()
s2 = Student()
```

s1 and s2 are objects created from the Student class.

3. Attributes (Variables inside a class)

Attributes store data related to an object.

```
Ex:

class Student:
    def __init__(self, name, age):
        self.name = name # attribute
        self.age = age   # attribute
```

```
s1 = Student("Sonali", 22)
print(s1.name)
print(s1.age)
```

4. Methods (Functions inside a class)

Methods define the behavior of an object.

```
Ex : class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"Hello, my name is {self.name}")

s1 = Student("Sonali")
s1.greet()
```

b. Difference between local and global variables.

- A. Understanding local vs global variables is important for controlling how data is accessed inside your Python programs.

- A **local variable** is defined **inside a function** and can be used **only within that function**.

■ Ex :

```
def my_function():
    x = 10 # local variable
    print(x)

my_function()
# print(x)
```

- A **global variable** is defined **outside all functions** and can be accessed anywhere in the program.

■ Ex :

```
x = 20 # global variable

def my_function():
    print(x)

my_function()

print(x)
```

7. Inheritance

a. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

A. Inheritance in Python is an OOP concept where one class (**child/derived class**) acquires properties and methods of another class (**parent/base class**). It helps in **code reuse** and better organization.

1. Single Inheritance

- One child class inherits from one parent class

```
class Parent:
    def show(self):
        print("This is parent class")
```

```
class Child(Parent):
    def display(self):
        print("This is child class")
```

```
obj = Child()
obj.show()
obj.display()
```

2. Multilevel Inheritance

- A class inherits from a child class (chain structure)

```
class Grandparent:
    def show1(self):
        print("Grandparent")
```

```
class Parent(Grandparent):
    def show2(self):
        print("Parent")
```

```
class Child(Parent):
    def show3(self):
        print("Child")
```

```
obj = Child()
obj.show1()
obj.show2()
obj.show3()
```

3. Multiple Inheritance

- One child class inherits from multiple parent classes

```
class Father:
    def skill1(self):
```

```
print("Driving")
```

```
class Mother:  
    def skill2(self):  
        print("Cooking")
```

```
class Child(Father, Mother):  
    def skill3(self):  
        print("Learning")
```

```
obj = Child()  
obj.skill1()  
obj.skill2()  
obj.skill3()
```

4. Hierarchical Inheritance

- Multiple child classes inherit from one parent class

```
class Parent:  
    def show(self):  
        print("Parent class")
```

```
class Child1(Parent):  
    pass
```

```
class Child2(Parent):  
    pass
```

```
obj1 = Child1()  
obj2 = Child2()
```

```
obj1.show()  
obj2.show()
```

5. Hybrid Inheritance

- Combination of two or more types of inheritance

```
class A:  
    def methodA(self):  
        print("Class A")
```

```
class B(A):  
    def methodB(self):  
        print("Class B")
```

```
class C(A):
```

```

def methodC(self):
    print("Class C")

class D(B, C): # Multiple + Hierarchical
    def methodD(self):
        print("Class D")

obj = D()
obj.methodA()
obj.methodB()
obj.methodC()
obj.methodD()

```

b. Using the super() function to access properties of the parent class

- A. In Python, the super() function is used to access methods and properties of a parent (base) class from a child (derived) class. It is especially useful in inheritance.

Why use super()?

- Avoids directly calling the parent class name
- Makes code cleaner and maintainable
- Supports multiple inheritance (MRO) properly

Ex :

```

class Parent:
    def show(self):
        print("This is parent class")

class Child(Parent):
    def show(self):
        super().show() # calling parent method
        print("This is child class")

obj = Child()
obj.show()

```

8. Method Overloading and Overriding

- a. **Method overloading: defining multiple methods with the same name but different parameters.**

- A. Method overloading means defining multiple methods with the same name but different parameters.

Python does NOT support traditional method overloading like languages such as Java or C++.

In Python, if you define multiple methods with the same name, the last definition overrides the previous ones.

Even though traditional overloading isn't supported, Python provides flexible ways to achieve similar behavior.

1. Default Arguments

```
class Demo:
    def add(self, a, b, c=0):
        return a + b + c
```

```
obj = Demo()
print(obj.add(2, 3))
print(obj.add(2, 3, 4))
```

2. Variable-Length Arguments (*args)

```
class Demo:
    def add(self, *args):
        return sum(args)
```

```
obj = Demo()
print(obj.add(2, 3))
print(obj.add(2, 3, 4, 5))
```

3. Type Checking Inside Method

```
class Demo:
    def add(self, a, b=None):
        if b is None:
            return a
        return a + b
```

```
obj = Demo()
print(obj.add(5))
print(obj.add(5, 10))
```

b. Method overriding: redefining a parent class method in the child class.

A. Method overriding means a **child class provides its own implementation** of a method that is already defined in its **parent class**.

It is used when you want to **change or extend the behavior** of a parent class method.

Ex : class Parent:

```
        def show(self):
```

```

        print("This is parent class method")

class Child(Parent):

    def show(self): # overriding

        print("This is child class method")

obj = Child()

obj.show()

```

9. SQLite3 and PyMySQL (Database Connectors)

- a. **Introduction to SQLite3 and PyMySQL for database connectivity.**
- A. Database connectivity lets your Python programs **store, retrieve, and manage data** efficiently. Two commonly used options are **SQLite3** and **PyMySQL**.

sqlite3 is a lightweight, file-based database that comes pre-installed with Python.

Features:

- i. No installation required
- ii. Stored as a **single file**
- iii. Best for **small projects and testing**

Example: Using SQLite3

```

import sqlite3

# Connect to database (creates file if not exists)
conn = sqlite3.connect("student.db")

# Create cursor
cursor = conn.cursor()

# Create table
cursor.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER, name TEXT)")

# Insert data
cursor.execute("INSERT INTO students VALUES (1, 'Sonali')")

# Save changes
conn.commit()

# Fetch data

```

```
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())
```

```
# Close connection
conn.close()
```

PyMySQL is a library used to connect Python with MySQL database server.

Features:

- Works with large-scale applications
- Supports multi-user environments
- Requires MySQL installation

Example :

```
import pymysql

# Connect to MySQL
conn = pymysql.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="test_db"
)

cursor = conn.cursor()

# Create table
cursor.execute("CREATE TABLE IF NOT EXISTS students (id INT, name VARCHAR(50))")

# Insert data
cursor.execute("INSERT INTO students VALUES (1, 'Sonali')")

# Save changes
conn.commit()

# Fetch data
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

# Close connection
conn.close()
```

b. Creating and executing SQL queries from Python using these connectors.

A. To work with databases in Python, you use connectors like SQLite3 and PyMySQL to create and execute SQL queries. The process is almost the same in both—connect → create cursor → execute queries → fetch results → close.

1. Steps to Execute SQL in Python

- Connect to database
- Create a cursor
- Execute SQL query
- Commit changes (if needed)
- Fetch results
- Close connection

2. Using SQLite3 (Built-in)

Example: Create Table, Insert, Fetch

```
import pymysql
```

```
# Step 1: Connect
```

```
conn = pymysql.connect(  
    host="localhost",  
    user="root",  
    password="your_password",  
    database="test_db"  
)
```

```
# Step 2: Cursor
```

```
cursor = conn.cursor()
```

```
# Step 3: Create table
```

```
cursor.execute("""  
CREATE TABLE IF NOT EXISTS students (  
    id INT,  
    name VARCHAR(50)  
)  
""")
```

```
# Step 4: Insert data
```

```
cursor.execute("INSERT INTO students (id, name) VALUES (%s, %s)",
(1, "Sonali"))
```

```
# Step 5: Commit
conn.commit()
```

```
# Step 6: Fetch data
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
```

```
for row in rows:
    print(row)
```

```
# Step 7: Close
conn.close()
```

10. Search and Match Functions

a. Using `re.search()` and `re.match()` functions in Python's `re` module for pattern matching.

A. Python's `re` module is used for regular expressions (pattern matching). Two commonly used functions are `re.match()` and `re.search()`.

- **`re.match()` Function**

- Checks for a match only at the beginning of the string
- Ex :

```
import re

text = "Python is easy"
result = re.match("Python", text)

if result:
    print("Match found")
else:
    print("No match")
```

- **`re.search()` Function**

- Searches for a pattern anywhere in the string
- Ex :

```
import re

text = "I love Python"
result = re.search("Python", text)
```

```
if result:
    print("Match found")
else:
    print("No match")
```

b. Difference between search and match

Feature	re.match()	re.search()
Position	Start only	Anywhere
Flexibility	Less	More
Returns	Match object or None	Match object or none
Common Use	Validate start of string	Find pattern anywhere